RELATÓRIO DE AULA

MÉTODO SOR - SOBRE-RELAXAÇÃO SUCESSIVA

Aluna: *Victoria Antonia Brito Barbosa*
Turma: *NB*
Curso: *Bacharelado em Ciência e Tecnologia - UNIFESP*
Disciplina: *Cálculo Numérico*

1. INTRODUÇÃO

Nesta aula de Cálculo Numérico, estudei o método SOR (Sobre-Relaxação Sucessiva), uma técnica iterativa utilizada para encontrar soluções aproximadas de sistemas de equações lineares. O método é uma extensão do método de Gauss-Seidel e tem como principal característica a utilização de um parâmetro de relaxação, que permite controlar as atualizações realizadas durante cada iteração.

O estudo envolveu a compreensão do funcionamento do algoritmo, a análise das condições que favorecem sua convergência e a implementação computacional em Python. Também foram realizados exercícios e experimentos com diferentes sistemas lineares, permitindo observar como a escolha dos parâmetros influencia a precisão e o desempenho do método.

2. FUNDAMENTAÇÃO TEÓRICA

Inicialmente, revisei o método de Gauss-Seidel, que resolve sistemas lineares por meio de aproximações sucessivas. Nesse procedimento, cada incógnita é atualizada utilizando os valores mais recentes disponíveis, fazendo com que as informações de uma mesma iteração sejam aproveitadas imediatamente nos cálculos seguintes.

A partir dessa ideia, foi apresentado o método SOR, que acrescenta um parâmetro denominado fator de relaxação, representado pela letra grega ômega (ω). Esse fator controla a intensidade da correção aplicada a cada nova

aproximação, podendo acelerar ou tornar mais lento o processo de convergência.

O comportamento do método depende do valor escolhido para ω. Quando o valor está entre 0 e 1, ocorre a sub-relaxação, na qual apenas uma parte da correção é aplicada. Quando ω é igual a 1, o método SOR se torna equivalente ao método de Gauss-Seidel. Já quando o valor está entre 1 e 2, ocorre a sobre-relaxação, que amplia a correção realizada durante cada iteração.

Compreendi que a escolha desse parâmetro é importante porque valores adequados podem reduzir a quantidade de iterações necessárias para encontrar uma solução, enquanto valores inadequados podem prejudicar a estabilidade do processo ou dificultar a convergência. Portanto, não existe um único valor de ω que seja necessariamente o melhor para todos os sistemas.

Também estudei algumas propriedades das matrizes que favorecem a convergência do método. Entre elas estão a diagonal dominante e a matriz simétrica definida positiva. Para analisar essa última propriedade, foi apresentado o critério de Sylvester, baseado na verificação dos menores principais da matriz.

Além disso, foram estudados os critérios de parada do algoritmo. A convergência pode ser acompanhada pela diferença entre duas aproximações consecutivas e pelo resíduo do sistema, que representa a diferença entre o vetor independente e o produto da matriz pelos valores aproximados encontrados. A utilização desses critérios permite avaliar se a solução obtida atende à precisão desejada. Também é estabelecido um número máximo de iterações para evitar que o algoritmo continue executando indefinidamente.

3. IMPLEMENTAÇÃO COMPUTACIONAL

A implementação do método SOR foi realizada em Python, utilizando o Google Colab. Foi desenvolvida uma função capaz de receber a matriz dos coeficientes, o vetor independente, o parâmetro de relaxação, a tolerância e o limite máximo de iterações.

Durante a execução, o algoritmo realiza as atualizações sucessivas das incógnitas e armazena os resultados de cada iteração. Ao final, retorna a solução aproximada, o histórico das aproximações, a quantidade de iterações realizadas e uma indicação sobre a ocorrência ou não da convergência.

A implementação também permite calcular o resíduo do sistema e acompanhar a diferença entre as aproximações consecutivas. Dessa forma, não é necessário analisar somente o resultado final, pois também é possível observar o comportamento do algoritmo durante todo o processo iterativo.

4. EXPERIMENTOS E ANÁLISE DOS RESULTADOS

Após a implementação, foram realizados experimentos para comparar o comportamento do método com diferentes valores do parâmetro de relaxação. Foram testados valores como 0,6, 0,9, 1,0, 1,3 e 1,6, permitindo observar as diferenças entre a sub-relaxação, o método de Gauss-Seidel e a sobre-relaxação.

A comparação foi realizada considerando principalmente a quantidade de iterações necessárias e o resíduo final. Também foram construídos gráficos para visualizar a redução do resíduo ao longo das iterações, facilitando a identificação de diferenças na velocidade de convergência.

Outro experimento envolveu a alteração da tolerância, utilizando valores de 10^{-4}, 10^{-8} e 10^{-12}. Com essa análise, observei que uma tolerância menor exige maior precisão da solução aproximada e pode aumentar a quantidade de iterações necessárias. Isso demonstra a importância de equilibrar a precisão desejada e o custo computacional.

Também foi realizada a comparação entre diferentes sistemas lineares, utilizando soluções de referência calculadas com a função numpy.linalg.solve. Essa comparação permitiu verificar a proximidade entre as aproximações obtidas pelo SOR e as soluções calculadas diretamente pelo NumPy.

5. EXERCÍCIOS E DESAFIOS EXTRAS

Os exercícios propostos envolveram a realização manual da primeira iteração do método SOR, a comparação entre diferentes valores de relaxação e a análise dos efeitos da alteração da tolerância. Essas atividades contribuíram para relacionar os cálculos teóricos com os resultados apresentados pelo algoritmo implementado.

Também trabalhei com a resolução de outros sistemas lineares, observando como as características das matrizes influenciam o comportamento do método. Nos desafios extras, foram analisados dois exemplos: um sistema que representa uma rede de quatro nós e outro que apresenta coeficientes e incógnitas em escalas diferentes.

Antes de executar os algoritmos, verifiquei a propriedade de diagonal dominante das matrizes. Em seguida, testei diferentes valores de ω e comparei a quantidade de iterações, os resíduos e as soluções aproximadas obtidas.

Nos exemplos analisados, o valor ω = 1,0 apresentou o menor número de iterações entre os parâmetros testados em alguns dos experimentos. Entretanto, também foi possível perceber que o desempenho não é igual para todos os sistemas. A eficiência depende das características da matriz e da escolha do parâmetro de relaxação, reforçando a necessidade de realizar testes e comparar os resultados.

6. CONCLUSÃO

Nesta aula, compreendi o funcionamento do método SOR e sua relação com o método de Gauss-Seidel, identificando como o parâmetro de relaxação modifica as atualizações realizadas durante o processo iterativo. Também estudei as condições que favorecem a convergência e os critérios utilizados para avaliar a precisão das soluções aproximadas.

A implementação em Python permitiu aplicar os conceitos estudados em diferentes sistemas lineares, comparar valores de relaxação, analisar tolerâncias e acompanhar graficamente a evolução dos resíduos. Os exercícios e desafios extras também contribuíram para compreender que o

comportamento do método depende das propriedades de cada matriz e que um parâmetro eficiente em determinado sistema pode não apresentar o mesmo desempenho em outro.

Concluo que o método SOR é uma alternativa importante para a resolução iterativa de sistemas lineares, principalmente quando se busca melhorar o desempenho do método de Gauss-Seidel. A aula permitiu relacionar a fundamentação matemática à implementação computacional, além de desenvolver minha capacidade de analisar a convergência e interpretar os resultados obtidos.